

Analyse d'une partie de billard 2025

Membres du groupe:

Makram Fadel : 372437

Kevin Jandri : 400096

Jamil Mansour : 381275

Rafaela Rebeiz : 377439

Plateforme: macOS

Compilateur employé: XCode, MatLab, LabVIEW

Noms des fichiers et leurs fonctions:

- **pixmap.bin** : fichier de données binaires contenant les pixels de l'image représentant la table de billard.
- **pos.txt** : fichier texte contenant les coordonnées des boules détectées ainsi que le score (*format : X Y score*) de chaque boule.
- **Pix2Pos** : fichier exécutable du programme C, appelé automatiquement depuis LabVIEW pour analyser l'image.
- **Pix2Pos.c** : le code source C.
- **AnalyseTx.m** : script MatLab généré par LabVIEW. *
- **SummaryTx.txt** : fichier texte contenant le résumé des résultats de la séquence. *
- **ScoreSheetTx.pdf** : fichier de sortie contenant la visualisation des trajectoires des boules, les collisions avec les bords du billard ainsi que l'indication du gagnant ou du perdant. *

* "**x**" représente le numéro de la séquence

SubVIs dans le LabVIEW:

- loop-proj.vi : détermination des dimensions du bord du billard
- call-C-proj.vi : exécution du code C dans LabVIEW
- script-Matlab-proj.vi : création du fichier AnalyseT(#séquence).m (Script MatLab)
- MP_LaunchMatlabScript4.vi : exécution du code Matlab

VI principal dans le LabVIEW:

- **Billard2025.vi** : VI principal du projet, liant les SubVIs entre-elles.

L'interface utilisateur est aussi gérée par ce VI.

Le flux de données

Pour chaque image de la séquence, LabVIEW convertit l'image en un fichier binaire (*Pixmap.bin*), tandis que les paramètres décrivant la table de billard sont transmis via la **ligne de commande**.

Ces paramètres sont ensuite fournis au programme C, dont le rôle est de détecter, pour chaque image, les positions successives des boules ainsi que leurs scores.

Les coordonnées calculées sont récupérées par LabVIEW, qui les mémorise et génère automatiquement un script MATLAB. Ce script contient les trajectoires des boules ainsi que les paramètres définis par l'utilisateur :

- le style graphique des trajectoires (**line**),
- la forme et la couleur des trajectoires des boules gagnante et perdante,
- le chemin du dossier de la séquence,
- deux options booléennes permettant de générer le script MATLAB et d'ouvrir automatiquement le PDF du score,
- la taille des boules (*ballsize*),
- les couleurs associées aux rebonds gagnants et perdants sur les bandes.

Le script MATLAB analyse ensuite la séquence afin de déterminer si le coup est gagnant ou pas. Il calcule également le nombre de bandes touchées, le nombre de boules ayant bougé, ainsi que la distance parcourue par chacune d'elles.

Enfin, deux fichiers de sortie sont générés :

- **SummaryTx.txt**, qui contient le résumé textuel des résultats,
- **ScoreSheetTx.pdf**, qui reprend ces informations et inclut en plus une analyse graphique complète de la séquence : trajectoire de la boule rouge en rouge, de la boule jaune en jaune et de la boule blanche en bleu. Les rebonds sur les bandes sont indiqués par des cercles, et les positions initiales des boules par des étoiles.

Descriptions gestion des erreurs:

C:

Erreurs:

1. **errLigneCommande** : signale un mauvais nombre d'arguments.
2. **errOuverture** : signale que l'ouverture des fichiers Pixmap.bin et/ou de Pos.txt n'est pas réussie.
3. **errPixelsManquants** : signale que le nombre de pixels de la Pixmap n'est pas exacte.
4. **errRectInvalide** : signale que les dimensions de la Pixmap ne sont pas conformes.
5. **errRectNegatif** : signale une dimension d'entrée négative.
6. **errColorRange** : signale un code RGB non dans les normes [0...255].
7. **errBallDiam** : signale que le diamètre de la boule est invalide (hors de la plage de valeurs attendue [10...15]).
8. **errMalloc** : signale l'échec de l'allocation dynamique.
9. **errTropPetite** : signale que le tableau a des dimensions inférieures à celles attendues
10. **errTropGrande** : signale que le tableau a des dimensions supérieures à celles attendues
11. **errBallLettre** : signale qu'un des caractères du paramètre du diamètre de la boule n'est pas conforme (lettre/symbole).

Dès qu'on rencontre l'une de ces erreurs, on affiche "**ERREUR: ...**" dans stderr (lisible par le LabView) et stoppe l'exécution, retournant ainsi le code respectif de l'erreur détectée.

Warning: L'exécution du code n'est pas interrompue.

Dans les cas où l'on ne détecte pas exactement 3 boules, on lui attribue systématiquement

$X = -1$, $Y = -1$, $Score = 0$.

Note: C'est notre seul warning. Le cas où on a trop de pixel est considéré comme une Erreur dans notre code (#10).

MATLAB :

Cette partie se focalise sur l'homogénéisation des données (par correction ponctuelle), afin d'assurer une lecture plus *smooth*.

1. **InterpolateNaN(X,Y)** : on détecte les positions contenant des NaN, puis les remplace en interpolant avec la prochaine valeur valide grâce à *interp1* avec l'option '*next*'.
2. **RemoveOutlier(X,Y)** : on identifie les valeurs aberrantes avec la méthode *movmedian* sur une fenêtre de taille 10, puis les remplace par la valeur précédente, après avoir déjà traité les NaN.

LABVIEW :

Le LABVIEW ne traite pas directement d'erreurs mais parvient à :

- lire un rectangle complet même dans le cas où la ligne/colonne de pixels du bord n'est pas strictement continue et à associer ses dimensions à celles du bord de la table de billard ;
- stopper le flux de donnée dans le cas où le fichier exécutable n'est pas lisible ;
- ignorer les fichiers avec une extension différente de .png ;
- détecter et relever une erreur s'il rencontre un mauvais fichier png (par exemple un fichier texte avec l'extension .png).

Son rôle est d'**afficher** et de relever les éventuelles erreurs détectées lors de son exécution ou de celle du code C.

Description succincte de nos algorithmes:

- Programme C

L'algorithme général du programme **Pix2Pos** se déroule en cinq grandes étapes :

Le programme commence par lire et vérifier les paramètres fournis par la ligne de commande (bornes du billard, plages de couleurs RGB et taille des boules).

Il ouvre ensuite le fichier binaire **pixmap.bin**, lit la largeur et la hauteur de l'image depuis la ligne de commande, puis charge tous les pixels en mémoire de manière dynamique.

Ensuite, il analyse l'image : pour chaque couleur de boule, il parcourt la zone intérieure du billard et calcule un score pour chaque carré de taille égale au diamètre de la boule, selon le nombre de pixels correspondant à la couleur recherchée. Le carré ayant le meilleur score indique la position de la boule.

Enfin, le programme enregistre les coordonnées et les scores des boules détectées dans le fichier **pos.txt**, puis libère la mémoire avant de se terminer.

Validation et tests

- Cas valides : fichiers complets et paramètres corrects → Pos.txt généré.
- Cas invalides : tests des erreurs attendues (fichier manquant, trop/pas assez de pixels, dimensions extrêmes, paramètres manquants...).
- Tests de robustesse : valeurs RGB hors limites, paramètres non valides, etc.

- Programme MATLAB

Le code commence par lire les coordonnées des boules fournies par le programme C via LabVIEW. À partir de ces données, il calcule les valeurs minimales et maximales afin de définir le cadre du billard, puis détermine les distances parcourues par chaque boule, l'indice de leur premier mouvement et l'ordre dans lequel elles commencent à bouger. Il détecte ensuite les rebonds de la première boule en mouvement contre les bandes du billard.

À l'aide de ces informations, le code analyse si la première boule ayant bougé gagne ou perd le point, en fonction du nombre de boules touchées et du nombre de bandes rencontrées avant le contact avec la dernière boule. Les résultats sont finalement sauvegardés dans **ScoreSheetTx.pdf** (analyse graphique) et **SummaryTx.txt** (résumé textuel).

Limitations du projet :

- 1) On prend normalement la distance initiale parcourue (ou le premier segment), le plus grand pour le cas où deux balles bougent aux mêmes indices. Mais dans le cas de la séquence T6, on a que White bouge à 3 et Blue bouge à 3 et les distances initiales parcourues sont **toutes les deux égales à 0**. Malgré cela, notre fonction trie correctement et détecte correctement que la boule jaune bouge en premier.
- 2) Dans les séquences T1 et T3, la FirstBall touche la LastBall et touche *ensuite* le bord du billard. Notre code détecte cependant l'événement inverse : **rebond avant contact avec la dernière boule**. Notre hypothèse est que cela est dû au fait que la fonction GetTouchIdx() retourne tous les indices où une boule touche une bande, et que nous conservons **le premier indice détecté**, même s'il est très proche temporellement d'un autre événement. Comme rien n'a été précisé à propos de ce petit problème, on a gardé cela tel quel.

Comment choisit-on l'encerclement des rebonds et le nombre de rebonds ?

Lorsque les trois boules bougent, seuls les rebonds compris entre le premier mouvement de la deuxième boule et celui de la troisième sont pris en compte, afin de ne conserver que les rebonds pertinents pour l'analyse. Le nombre de bandes correspond alors au nombre d'indices retenus dans la variable **touch**.

Si moins de trois boules bougent, aucun filtrage temporel n'est appliqué : tous les rebonds détectés sont conservés et leur nombre est utilisé tel quel dans *ScoreSheetTx.pdf* que dans *SummaryTx.txt*. On aurait voulu lui assigner *N/A* ou autre, mais comme rien n'était précisé à ce sujet, on a simplement gardé **touch** tel quel, sans aucun filtrage.

- Utilisation de **sortrows()**
On trie d'abord toutes les lignes selon les valeurs de la colonne 1. En cas d'égalité, seules les lignes concernées sont ensuite triées selon la colonne 2, en ordre décroissant.
- Variable **'moved_idx'**
La colonne 1 étant déjà triée, on crée un vecteur logique. Ce vecteur permet de sélectionner uniquement certaines cases d'une autre colonne. Par exemple, [true, false, true] permet de garder [case 1, case 3].
- fonction **GetBallMoveOrder()**:
On aurait pu employer plein de *if* et de *elseif* pour différencier et tester chaque cas, mais on a opté pour une méthode qui met en valeur un des atouts clés de MatLab : le traitement efficace des tableaux sans logique conditionnelle compliquée.

- Programme LabVIEW

On lit le dossier de la séquence contenant les images de la partie de billard dans le **Main VI**, ainsi que le nombre d'éléments (file **.png**) dans ce dossier. À partir de ça, on crée une pixmap (qui traduira l'image en pixels RGB sous forme binaire) pour la première image lue, qui sera envoyée dans le SubVI **loop-proj.vi**. On trouve ensuite les dimensions du bord de la table en lisant chaque ligne et colonne de pixels retenant les RGB représentant du Dark Blue.

Ensuite, on exécute une For Loop afin d'obtenir une pixmap.bin pour chaque image.

Le code C sera par la suite exécuté depuis la sub VI **call C proj.vi**.

Enfin, les données ci-dessus sont envoyées au subVI **script-Matlab-proj.vi**, qui va créer le fichier *AnalyseTx.m* contenant le reste des informations requises, telles que les commandes tirées du contrôle panel modifié par l'utilisateur-même. On exécute le code MatlaB à partir du subVI **MP_LaunchMatlabScript4.vi** qui génère les deux fichiers: *SummaryTx.txt* et *ScoreSheetTx.pdf*.

Remarque, comme demandée: Problème possible lié à l'exécutable Pix2Pos:

Lors de l'exécution du projet sur un autre ordinateur, il est possible qu'une erreur (**code 126 – ./ Pix2Pos Operation not permitted**) apparaisse dans LabVIEW. Ce problème est lié aux protections du système et ne nécessite aucune modification du code : il peut être résolu lors de l'évaluation si nécessaire grâce à une recompilation manuelle ou à une désactivation du système de protection de l'OS.